Data Structures

BS (CS) _Fall_2025

Lab_06 Manual

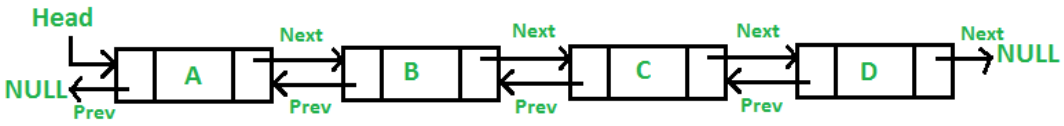
Learning Objectives:

1. Linked Lists (Doubly, Circular)

Doubly Linked List in C++

A doubly linked list is a type of linked list in which each node consists of 3 components:

- *prev - address of the previous node
- data - data item
- *next - address of next node




```
struct node {
    int data;
    struct node *next;
    struct node *prev;
}
```

Advantages of Doubly Linked List in C++

- The doubly linked list allows us to traverse in both forward and backward directions.
- The deletion of a node is more efficient and easy as it has a pointer to the previous node.
- It is easier to perform the reverse operation.
- A doubly linked list is dynamic in nature so it can grow or shrink in size.

Disadvantages of Doubly Linked List in C++

- It requires more memory than arrays, per node due to the additional storage used by the pointers.
- It is more complex to implement and maintain as compared to the singly-linked list.
- We have to traverse from the head node to the specific node for insertion and deletion at specific positions.

Insertion in Doubly Linked List in C++

In a doubly linked list, there are three ways in which the insertion operation can be performed:

- Insertion at the beginning of DLL
- Insertion at the end of DLL
- Insertion at Specific Position in DLL

Deletion in Doubly Linked List in C++

Like insertion, there are three ways in which the deletion operation can be performed:

- Deletion at the beginning of DLL
- Deletion at the end of DLL
- Deletion at Specific Position in DLL

Traversal in a Doubly Linked List in C++

Traversing a doubly linked list means iterating through the list by visiting each node and performing the desired operations. This can be done by maintaining a temp variable that starts from the head of the doubly linked list and follows the next pointer for traversing until the end of the list. In a doubly linked list, we can traverse in both directions:

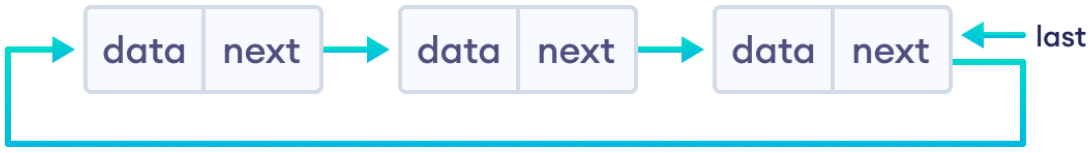
- In forward traversal, traverse from the head to the tail node.
- In reverse traversal, traverse from the tail to the head node.

Circular Linked List in C++

A circular linked list is a type of linked list in which the first and the last nodes are also connected to each other to form a circle.

There are basically two types of circular linked list:

- Circular Singly Linked List
- Circular Doubly Linked List




```
struct Node {
    int data;
    struct Node * next;
};
```

Few Advantages

- The NULL assignment is not required because a node always points to another node.
- The starting point can be set to any node.
- Traversal from the first node to the last node is quick.

Applications of Circular Linked List

Few are some of the applications of Circular Linked Lists:

- Circular Buffers (Ring Buffers)
- Round-Robin Scheduling in CPU
- Implementing Undo Functionality
- Music Playlists